

MODUL PRAKTIKUM #3

10S2203 - Sistem Operasi

Linux's Inter-process Communication dengan Pipe

Minggu	:	11
Setoran	:	• Lembar jawaban • Program • Laporan
Batas Waktu Setoran	:	• Bagian A. Teori dan penjelasan B. Pemrograman (Prosedur 2 No 3) [End of Session] • Source Code dan Laporan Praktikum [Rabu, 22 April 2026]
Tujuan	:	1. Mampu menjelaskan konsep-konsep dasar komunikasi antar proses (<i>inter-process communication – IPC</i>) 2. Mampu menulis program untuk komunikasi antar proses pada Linux menggunakan <i>pipe</i> dan <i>named pipe</i>.

Petunjuk Praktikum:

- Anda dapat mengerjakan praktikum ini secara berkelompok dengan maksimum 3 mahasiswa per kelompok. Anda bebas memilih anggota kelompok.
- Setiap kelompok harus menyerahkan setoran sesuai dengan batas waktu setoran. Pada laporan harus dicantumkan identitas anggota kelompok seperti NIM, Nama dan kontribusi dalam mengerjakan tugas ini, seperti contoh

NIM	Nama	Kontribusi
11S20003	Reinhard	Mengerjakan soal nomor 1 dan 2
11S20022	Peter	Mengerjakan soal nomor 3, dan 4
11S20034	Kevin	Mengerjakan soal nomor 5

- Anda hanya diperbolehkan diskusi dengan sesama anggota kelompok. Mencontoh pekerjaan dari kelompok lain dianggap plagiarisme dan akan ditindak sesuai dengan sanksi akademik yang berlaku di IT Del.
- Keterlambatan menyerahkan laporan tidak ditolerir dengan alasan apapun. Oleh karena itu, laporan harus dikumpulkan tepat waktu.
- Gunakan sistem operasi Linux boleh menggunakan Distro apapun namun untuk mempermudah praktikum, disarankan menggunakan Ubuntu.

Referensi:

1. A. Silberschatz, P.B. Galvin, and G. Gagne, Operating System Concepts, 9th edition, Chapter 1 and 2, John Wiley & Sons, Inc., 2013.
2. M. Neil, S. Richard, Beginning Linux Programming, 4th edition, Wiley, 2008.
3. Geeksforgeeks.org, Named Pipe or FIFO with example C program, <
<https://www.geeksforgeeks.org/named-pipe-fifo-example-c-program/>> [diakses pada 20 Maret 2020].

Pengantar

Pada minggu ke-3 saat sesi kelas tatap muka dengan topik: *Process Management*, anda telah mempelajari bahwa proses-proses yang membutuhkan kerja sama (*cooperating processes*) memerlukan sebuah teknik/metode komunikasi. Anda juga telah paham bahwa perancang sistem operasi telah menetapkan bahwa setiap proses yang sedang aktif, memiliki ruang alamat memori masing-masing yang bersifat pribadi (*virtual private address space*) yang dialokasikan oleh sistem operasi. Dengan demikian, sebuah proses tidak dapat mengakses memori dari proses lain. Pada sisi lain, sangat lazim sebuah proses berinteraksi dengan proses lain dalam melakukan tugasnya. Karena proses-proses yang saling bekerja sama tersebut tidak dapat mengakses memori satu sama lainnya, maka disediakan 2 model atau teknik untuk komunikasi antar proses, yakni: *message passing* dan *shared memory*. Saat ini, anda mempraktikkan mekanisme baku di lingkungan Sistem Operasi Linux untuk komunikasi antar proses dengan *pipe* yang dikelompokkan ke dalam model *shared memory*. Linux mendukung teknik ini dengan memberikan fungsi-fungsi seperti *pipe()*, *read()*, dan *write()*.

Sementara itu, model komunikasi antar proses dengan *message passing* dapat diterapkan dengan *message queue*. Model ini lazim diterapkan di komunikasi klien-server yang dicirikan dengan adanya jalur komunikasi (*communication link*) antara pengirim (*sender*) dan penerima (*receiver*). Pemahaman dan pembentukan kemampuan praktikal untuk *message queue* diserahkan kepada anda sebagai pembelajaran mandiri.

A. [40 poin] Teori

1. **[12 poin]** Jelaskan 4 alasan berikut yang menjadi dasar dibutuhkan komunikasi antar proses: *information sharing*, *computation speedup*, *modularity*, dan *convenience*.
2. **[8 poin]** Sebutkan 2 contoh penerapan komunikasi antar proses yang lazim terjadi di komputer.
3. **[10 poin]** Jelaskan karakteristik komunikasi antar proses dengan model *message passing*.
4. **[10 poin]** Jelaskan keuntungan dan kerugian dari komunikasi antar proses yang menerapkan teknik-teknik berikut:
 - a. *synchronous vs asynchronous communication*,
 - b. *fixed-sized* dan *variable-sized messages*.

B. [30 poin] Pemrograman

1. Anonymous pipe

Dalam Sistem Operasi UNIX, *pipes* berguna untuk komunikasi antara proses terkait (*inter-process communication*). *Pipe* adalah komunikasi satu arah saja dimana kita dapat menggunakan *pipe* sedemikian hingga satu proses menulis ke *pipe*, dan proses lainnya membaca dari *pipe*. *Pipe* merupakan area memori utama yang diperlakukan sebagai "file virtual" dan dapat diakses oleh proses-proses yang bekerja sama. Dengan demikian, *pipe* melonggarkan batasan ruang alamat memori yang bersifat pribadi terhadap proses. *Pipe* dapat digunakan oleh proses yang membuat *pipe* tersebut serta semua proses anaknya, untuk membaca dan menulis sehingga lazim digunakan untuk komunikasi proses induk dan anak. Satu proses dapat menulis ke "file virtual" atau *pipe* ini dan proses lainnya dapat membaca

darinya. Jika suatu proses mencoba membaca sebelum ada sesuatu yang ditulis ke *pipe*, proses itu ditunda sampai ada sesuatu yang ditulis. Pemanggilan sistem *pipe* akan menemukan dua posisi pertama yang tersedia dalam tabel berkas proses yang sedang dibuka (*per-process open-file table*) dan mengalokasikannya untuk ujung *pipe* baca dan tulis.

```
int pipe(int fds[2]);
```

Parameters :

fd[0] will be the fd(file descriptor) for the read end of pipe.

fd[1] will be the fd for the write end of pipe.

Returns : 0 on Success.

-1 on error.

Prosedur 1:

1. Buatlah berkas dengan nama `pipe_basic.c` lalu tambahkan kode di bawah ini pada berkas tersebut.

```

1
2  #include <stdio.h>
3  #include <unistd.h>
4  #define MSGSIZE 16
5  char* msg1 = "hello, world ";
6
7
8  int main()
9  {
10     char inbuf[MSGSIZE];
11     int p[2], i;
12
13     if (pipe(p) < 0)
14         exit(1);
15
16     /* continued */
17     /* write pipe */
18
19     write(p[1], msg1, MSGSIZE);
20
21
22     /* read pipe */
23     read(p[0], inbuf, MSGSIZE);
24     printf("%s\n", inbuf);
25
26     return 0;
27 }
```

2. Pada baris ke-13 terdapat pengecekan keberhasilan dari pembuatan *pipe* berdasarkan dari nilai *return*-nya. Jika nilainya kurang dari 0 maka pembuatan *pipe* gagal, dan jika nilainya sama dengan 0 maka pembuatan *pipe* berhasil.
3. Pada baris ke-19 terdapat fungsi *write()* untuk menuliskan data pada *pipe* yang telah dibuat. Perhatikan fungsi tersebut pada baris ke-19.

```
19 write(p[1], msg1, MSGSIZE);
```

Parameter pertama dari fungsi di atas adalah *fd* (*file descriptor*) dimana `p[1]` merupakan *fd* untuk *write end* pada *pipe*. Parameter kedua adalah buffer untuk menuliskan data. Data

yang akan ditulis pada buffer diisi dengan variabel `msg1`. Parameter terakhir merupakan panjang dari buffer tersebut.

4. Pada baris ke-23 terdapat fungsi `read()` untuk membaca data pada *pipe* yang telah dibuat. Perhatikan fungsi tersebut pada baris ke-23.

```
23      read(p[0], inbuf, MSGSIZE);
```

Parameter pertama dari fungsi di atas adalah *fd* (*file descriptor*) dimana `p[0]` merupakan *fd* untuk *read end* pada *pipe*. Parameter kedua adalah buffer untuk membaca data. Parameter terakhir merupakan panjang dari buffer tersebut.

Prosedur 2:

Berikut adalah program Linux C untuk membuat dua proses P1 dan P2. P1 mengambil string dan meneruskannya ke P2. P2 menggabungkan string yang diterima dengan string lain tanpa menggunakan fungsi string dan mengirimkannya kembali ke P1 untuk dicetak.

1. Buatlah berkas dengan nama `pipe_fork.c` lalu tambahkan kode di bawah ini pada berkas tersebut.

```
1  #include<stdio.h>
2  #include<stdlib.h>
3  #include<unistd.h>
4  #include<sys/types.h>
5  #include<string.h>
6  #include<sys/wait.h>
7
8  int main()
9  {
10     int fd1[2];
11     int fd2[2];
12
13     char fixed_str[] = "Interprocess communication";
14     char input_str[100];
15     pid_t p;
16
17     if (pipe(fd1)==-1)
18     {
19         fprintf(stderr, "Pipe Failed" );
20         return 1;
21     }
22     if (pipe(fd2)==-1)
23     {
24         fprintf(stderr, "Pipe Failed" );
25         return 1;
26     }
```

```

27     scanf("%s", input_str);
28     p = fork();
29
30     if (p < 0)
31     {
32         fprintf(stderr, "fork Failed" );
33         return 1;
34     }
35
36     // Parent process
37     else if (p > 0)
38     {
39         char concat_str[100];
40
41         close(fd1[0]);
42
43         write(fd1[1], input_str, strlen(input_str)+1);
44         close(fd1[1]);
45
46         wait(NULL);
47
48         close(fd2[1]);
49
50         read(fd2[0], concat_str, 100);
51         printf("Concatenated string %s\n", concat_str);
52         close(fd2[0]);
53     }
54     // child process
55     else
56     {
57         close(fd1[1]);
58
59         char concat_str[100];
60         read(fd1[0], concat_str, 100);
61
62         int k = strlen(concat_str);
63         int i;
64         for (i=0; i<strlen(fixed_str); i++)
65             concat_str[k++] = fixed_str[i];
66
67         concat_str[k] = '\0';
68
69         close(fd1[0]);
70         close(fd2[0]);
71
72         write(fd2[1], concat_str, strlen(concat_str)+1);
73         close(fd2[1]);
74
75         exit(0);
76     }
77 }

```

2. Berikut adalah hasil dari program di atas setelah dijalankan.

```
Concatenated string SISOP-Interprocess communication
```

3. **[5 poin]** Jelaskan cara kerja *pipe* yang terdapat pada *parent* dan *child process*!
4. **[10 poin]** Modifikasilah program di atas sehingga hasilnya menampilkan nama/nim-prodi anda masing-masing! Simpan program sebagai `pipe_fork_updated.c`.

2. Named Pipe

Named Pipe memiliki fitur-fitur yang membuatnya jauh lebih manjur daripada *anonymous pipe*. Fitur-fitur tersebut dijelaskan sebagai berikut:

- i. *Named pipe* merupakan mekanisme *message-oriented*, yang berarti proses membaca *pipe* (*reading process*) dapat membaca pesan yang tidak sama panjangnya (*variabel length messages*) dengan yang ditulis oleh proses penulis (*writing process*).
- ii. *Named pipe* adalah bi-direksional (*full duplex*) sehingga 2 proses dapat bertukar pesan menggunakan 1 *pipe* yang sama.
- iii. Sebuah *pipe* dapat memiliki beberapa instan dengan nama yang sama.
- iv. Klien dalam sebuah jaringan (*network clients*) dapat mengakses *named pipe*. Komunikasi *named pipe* adalah sama untuk proses-proses yang ada pada komputer yang sama maupun proses-proses yang berada pada jaringan atau komputer yang berbeda.
- v. Linux telah membuat fungsi-fungsi yang nyaman (*convenience function*) yang dapat digunakan dengan lebih sederhana dan lebih mudah untuk pengelolaan *named pipe*.

Aturan yang dapat anda gunakan untuk memutuskan kapan menggunakan *anonymous pipe* dan kapan menggunakan *named pipe* adalah sebagai berikut:

- i. Gunakanlah *anonymous pipe* untuk mengarahkan I/O pada *cooperating processes* yang ada pada komputer yang sama dan memerlukan hubungan proses induk dan anak.
- ii. Sedangkan jika anda membutuhkan komunikasi *full-duplex*, *messageoriented*, dan *cooperating processes* berada pada jaringan atau komputer yang berbeda, maka gunakanlah *named pipe*.

Biasanya *named pipe* muncul sebagai berkas dan umumnya proses melampirkannya untuk komunikasi antar-proses. Berkas *FIFO* adalah jenis khusus pada penyimpanan lokal yang memungkinkan dua atau lebih proses untuk berkomunikasi satu sama lain dengan membaca / menulis ke / dari berkas ini. Berkas khusus *FIFO* dimasukkan ke dalam sistem berkas dengan memanggil *mkfifo()* di C. Setelah kita membuat berkas khusus *FIFO* dengan cara ini, setiap proses dapat membukanya untuk membaca atau menulis, dengan cara yang sama seperti berkas biasa. Namun, itu harus terbuka di kedua ujung secara bersamaan sebelum Anda dapat melanjutkan untuk melakukan operasi input atau output di atasnya.

Untuk membuat berkas *FIFO* kita gunakan fungsi berikut:

```
int mkfifo(const char *pathname, mode_t mode);
```

mkfifo() membuat file khusus *FIFO* dengan nama *pathname*. Sedangkan *mode*, menentukan *permission* dari *FIFO* tersebut.

Berikut adalah contoh program untuk mengilustrasikan *named pipe*. Ada dua program yang menggunakan *FIFO* yang sama. Program 1 (*pipe_wr.c*) menulis terlebih dahulu, lalu membaca. Program 2 (*pipe_rw.c*) membaca terlebih dahulu, lalu menulis. Kedua program tersebut akan terus melakukannya sampai diakhiri.

Prosedur

1. Buatlah berkas dengan nama *pipe_wr.c* lalu tambahkan kode di bawah ini pada berkas tersebut.

```

1  #include <stdio.h>
2  #include <string.h>
3  #include <fcntl.h>
4  #include <sys/stat.h>
5  #include <sys/types.h>
6  #include <unistd.h>
7
8  int main()
9  {
10     int fd;
11     char * myfifo = "/tmp/myfifo";
12     mkfifo(myfifo, 0666);
13     char arr1[80], arr2[80];
14     while(1)
15     {
16         fd = open(myfifo, O_WRONLY);
17         fgets(arr2, 80, stdin);
18         write(fd, arr2, strlen(arr2)+1);
19         close(fd);
20         fd = open(myfifo, O_RDONLY);
21         read(fd, arr1, sizeof(arr1));
22         printf("User2: %s\n", arr1);
23         close(fd);
24     }
25     return 0;

```

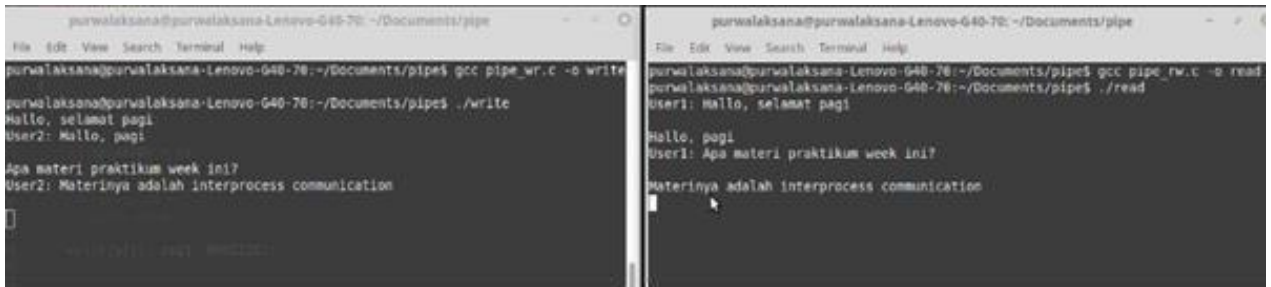
2. Buatlah berkas dengan nama `pipe_rw.c` lalu tambahkan kode di bawah ini pada berkas tersebut.

```

1  // This side reads first, then reads
2  #include <stdio.h>
3  #include <string.h>
4  #include <fcntl.h>
5  #include <sys/stat.h>
6  #include <sys/types.h>
7  #include <unistd.h>
8
9  int main()
10 {
11     int fd1;
12     char * myfifo = "/tmp/myfifo";
13     mkfifo(myfifo, 0666);
14     char str1[80], str2[80];
15     while (1)
16     {
17         fd1 = open(myfifo, O_RDONLY);
18         read(fd1, str1, 80);
19         printf("User1: %s\n", str1);
20         close(fd1);
21         fd1 = open(myfifo, O_WRONLY);
22         fgets(str2, 80, stdin);
23         write(fd1, str2, strlen(str2)+1);
24         close(fd1);
25     }
26     return 0;
27 }

```


3. Berikut adalah hasil setelah kedua program di atas dijalankan.



```
purwalaksana@purwalaksana-Lenovo-G40-70: ~/Documents/pipe
File Edit View Search Terminal Help
purwalaksana@purwalaksana-Lenovo-G40-70:~/Documents/pipe$ gcc pipe_wr.c -o write
purwalaksana@purwalaksana-Lenovo-G40-70:~/Documents/pipe$ ./write
Halo, selamat pagi
User2: Halo, pagi
Apa materi praktikum week ini?
User2: Materinya adalah interprocess communication

purwalaksana@purwalaksana-Lenovo-G40-70: ~/Documents/pipe
File Edit View Search Terminal Help
purwalaksana@purwalaksana-Lenovo-G40-70:~/Documents/pipe$ gcc pipe_rw.c -o read
purwalaksana@purwalaksana-Lenovo-G40-70:~/Documents/pipe$ ./read
User1: Halo, selamat pagi
Halo, pagi
User1: Apa materi praktikum week ini?
Materinya adalah interprocess communication
```

4. [15 poin] Jelaskan cara kerja *named pipe* pada kedua program di atas sehingga setelah kedua program tersebut dijalankan hasilnya seperti pada poin 3!

C. [30 poin] Tugas Pemrograman

Buatlah 2 program yang saling bekerja sama menggunakan Named Pipe (FIFO) untuk mensimulasikan system transaksi kasir dengan spesifikasi sebagai berikut:

- a. Program pertama, `master.c` (Simulasi pelanggan/ Input) adalah program yang bertugas:
 - i. Input data: membuat sebuah array atau sekumpulan data yang berisi harga barang bertipe *integer* yang dibeli oleh pelanggan. Jumlah barang diserahkan kepada anda (minimal 5 barang);
 - ii. menulis daftar harga tersebut ke *named pipe* untuk dibaca oleh program ke-2;
 - iii. menunggu proses perhitungan total dan diskon yang dilakukan oleh program ke-2 (lihat butir b); dan
 - iv. membaca hasil akhir total belanja dari program ke-2 serta mencetaknya ke monitor.
- b. Program kedua, `worker.c` adalah simulasi mesin kasir/ Processing program yang bertugas:
 - i. membaca daftar harga barang yang ditulis oleh `master.c` ke *named pipe*;
 - ii. Proses Data:
 - Menghitung total harga belanjaan.
 - **Spesifikasi Tambahan (opsional):** Coba berikan diskon sebesar 10% jika belanjaan melebihi Rp100.000; dan
 - iii. menulis hasil perhitungan (Total kotor, Diskon, dan Total Bersih) oleh program ke-2 ke *named pipe* untuk dibaca oleh `master.c`.

Spesifikasi Teknis:

- Kedua proses harus menggunakan berkas FIFO yang sama (misal: `/tmp/kasir_fifo`).
- Gunakan fungsi `mkfifo()` untuk membuat pipa dan fungsi `open()`, `read()`, `write()`, serta `close()` untuk manajemen datanya.
- Mahasiswa diperbolehkan menggunakan *struct* atau *string formatting* (menggunakan `sprintf`) untuk mengirim data yang kompleks melalui pipa.
- Anda dapat menambahkan spesifikasi lain yang perlu.